Dino Game Bot — Brief

A pixel-vision autoplayer with a live status dashboard

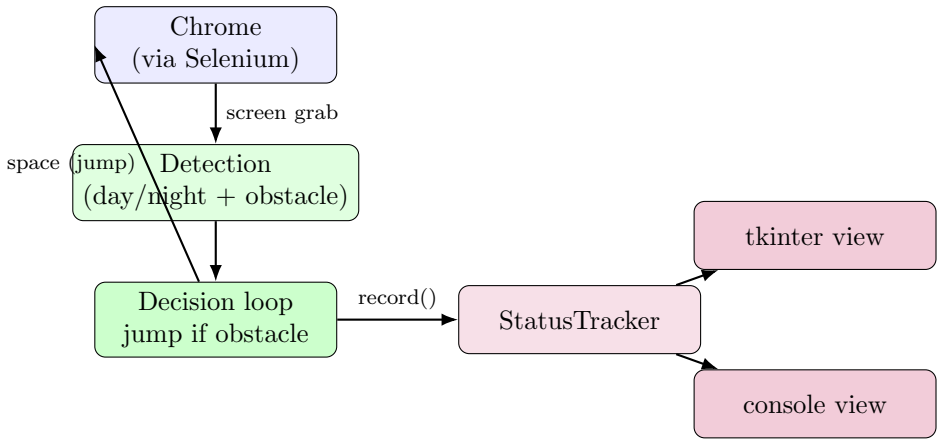
Explainer document generated for the project owner

Concise version of the full Deep Dive: same facts, higher altitude, key diagrams only. See `deep-dive.pdf` for line-by-line detail.

1 What it does

A single Python script (`main.py`) plays Chrome's offline Dino game autonomously. It never touches the game's DOM or internal state; it only reads screen pixels in two fixed rectangular regions and presses space when it sees an obstacle color where the dino is about to be. A small live status window (or a console fallback when no display exists) shows what the loop is currently seeing, as an observer layered on top of the original detection logic.

2 Architecture at a glance



Two independent layers share one loop: **detection/play** (unchanged from the original script, does not know a dashboard exists) and **status reporting** (a pure state object plus a swappable view, purely observational – it cannot affect whether the bot jumps).

3 Detection logic

- **Day/night check** (`check_day`): grabs a large screen region, counts white vs. dark-grey pixels, and majority-votes the current color mode. Needed because the game periodically inverts its palette, which flips which color counts as "obstacle."
- **Obstacle check** (`check_obstacle`): grabs a narrow box just ahead of the dino and checks whether *any* pixel matches the obstacle color for the current mode. No shape, size, or distance

estimate — presence alone triggers a jump.

- **Loop**: on every ∼10 ms iteration, if an obstacle is present, send a space keypress immediately, then feed the iteration's results to the status tracker and view. Exits on the `q` key or the status window's close button.

Both region boxes are absolute screen-pixel coordinates tuned for one specific maximized-window/resolution combination; they need re-measuring on any other machine.

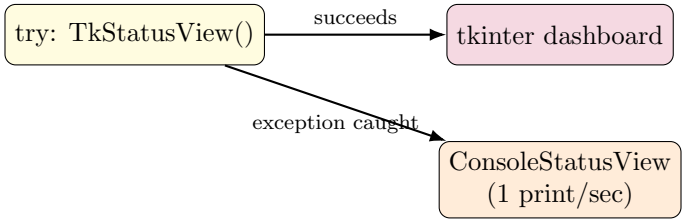
4 Status dashboard

`StatusTracker` is a plain, display-independent object updated once per loop iteration with the mode, obstacle flag, and whether a jump was sent; it tracks a running jump count, elapsed time, and a rolling log of the last 8 events. Being GUI-free, it can be tested purely as logic.

Two interchangeable views read that state:

- `TkStatusView`: a small tkinter window with live labels and an event log. It never calls tkinter's blocking `mainloop()`; instead it does a manual, non-blocking refresh once per loop iteration so the GUI and the ∼10 ms polling loop coexist on one thread.
- `ConsoleStatusView`: prints a throttled status line (at most once per second) to stdout, used automatically whenever no graphical display is available.

`build_status_view()` tries to construct the tkinter window first and catches any exception (no display, Tk missing, headless CI) to fall back to the console view, so the bot never crashes purely because a GUI could not be created.



5 Dependencies

`selenium` drives Chrome; `Pillow` provides the screen-capture calls the detection logic reads; `keyboard` provides the global `q`-to-quit hotkey (needs elevated privileges on Linux since it reads the input device directly). `tkinter` is standard-library and not listed separately.

6 Key trade-offs (from the README)

- Coordinates are hardcoded to one screen; not derived from actual window geometry.
- Day/night detection costs an extra screen grab per loop iteration (mitigated slightly by the dashboard reusing that result rather than recomputing it).
- A known ChromeDriver quirk (exception on navigating to `chrome://dino/` that doesn't actually indicate failure) is deliberately swallowed.
- Obstacle detection is presence-only, with no distance/speed estimate or jump-duration tuning.

7 Verification status

Verified without a live game: detection functions against synthetic images, `StatusTracker` state transitions, `TkStatusView` against a real X display, the headless fallback with `DISPLAY` unset, and a clean dependency install. Not verified: an actual Chrome run against the live game, whether the hardcoded coordinates transfer to another screen, or whether the dashboard refresh introduces reaction-time lag at full game speed.

8 License